TensorConversion.h & TensorConversion.c

Full Explanation — Concepts · Quantization Math · Every Function Line by Line

Colour legend:

- Function signature / struct
- Variable declaration
- Logic / loop / condition
- Return statement
- Guard / error check
- Comment

1 — Key Concepts

Quantization: Compressing a floating-point number into a smaller integer by dividing by a scale factor. Instead of storing 0.00394 (32 bits), you store 127 (8 bits) and remember scale=0.000031. To recover the float: integer × scale ≈ original.
Analogy: Measuring a room in centimetres instead of millimetres — fewer digits, small loss of precision.

Scale: A float stored in qConfig->scale. It is the conversion factor between the integer world and the float world. real_value = mantissa × scale. Choosing the right scale determines how much precision is lost.
Analogy: The zoom level on a map — 1cm on the map = 1km in reality.

Mantissa: The stored integer value in a quantized tensor. It has no unit by itself — multiply it by scale to get the real value.
Analogy: The number on a map. Meaningless without the map's scale.

qMax / qMin: The largest and smallest integers that can be stored for a given bit width. For 8-bit symmetric: qMax=127, qMin=-128. For 16-bit: qMax=32767, qMin=-32768. Formula: qMax = 2^(qMaxBits-1) - 1, qMin = -2^(qMaxBits-1).
Analogy: The maximum and minimum values on a thermometer scale.

Symmetric quantization (SYM / SYM_INT32): The range of representable values is symmetric around zero: [-qMax, +qMax]. Only one parameter needed: scale. Good for weights (which are naturally centred around 0). SYM_INT32 stores mantissas as 32-bit integers (large range, used in FQT training). SYM stores as N-bit packed integers (e.g. 8-bit, more compact).
Analogy: A ruler with 0 in the middle and equal marks left and right.

Asymmetric quantization (ASYM): The range can be shifted: [min, max] maps to [0, qMax]. Needs two parameters: scale AND zeroPoint. real_value = (integer + zeroPoint) × scale. Good for activations (which are often non-negative after ReLU).
Analogy: A ruler that starts at an arbitrary non-zero value.

zeroPoint: An integer offset used in asymmetric quantization. It shifts the zero of the float range to a specific integer. real_value = (integer + zeroPoint) × scale.
Analogy: The offset on a thermometer that starts at -40 instead of 0.

clamp(x, min, max): Forces x to be within [min, max]. If x < min, returns min. If x > max, returns max. Otherwise returns x.
Analogy: A volume knob that cannot go below 0 or above 100.

roundByMode(x, mode): Rounds a float to an integer using either HALF_AWAY (deterministic, like standard rounding) or SR_HALF_AWAY (stochastic — adds a random jitter before rounding, used in FQT training to eliminate bias).
Analogy: Rounding 1.5 to 2 always (HALF_AWAY) vs. randomly to 1 or 2 (SR_HALF_AWAY).

Function pointer type conversionFunction_t: A data type that holds the address of a function. typedef void (*conversionFunction_t)(tensor_t*, tensor_t*) means: any variable of this type stores the address of a function that takes two tensor_t* and returns void.
Analogy: A variable that holds a phone number — you call whoever's number is stored there.

conversionMatrix[6][6]: A 6×6 table of function pointers. Row = source type, Column = destination type. conversionMatrix[FLOAT32][SYM_INT32] holds the address of convertFloatTensorToSymInt32Tensor. convertTensor uses this table to find the right function automatically.
Analogy: A train timetable — row=origin station, column=destination, cell=which train to take.

In-place operation: When inputTensor and outputTensor point to the same tensor (same data buffer). Safe only if the algorithm reads each element before writing it. Saves memory — no extra buffer needed.
Analogy: Editing a document in place vs. making a copy to edit.

memmove vs memcpy: Both copy bytes. memcpy is faster but UNSAFE if source and destination overlap. memmove handles overlapping regions correctly. convertTensorsWithSameType uses memmove precisely because input and output may be the same buffer.
Analogy: memcpy = carrying boxes in a line. memmove = using a forklift that handles overlaps.

_Static_assert: A compile-time check. If the condition is false, the compiler refuses to compile and prints the message. _Static_assert(BOOL+1==6, ...) ensures the conversionMatrix is always 6×6 — if someone adds a new qtype_t without extending the matrix, compilation fails immediately.
Analogy: A pre-flight checklist that the pilot must complete before the plane can take off.

VLA — Variable Length Array: An array whose size is determined at runtime, not compile time. int32_t data[n] where n is a variable. Allocated on the stack. Fine for small n; dangerous for large n on STM32 (small stack).
Analogy: A bag that expands to exactly the size of what you put in it.

fabsf(x): Standard C math: returns the absolute value of a float. fabsf(-3.5) = 3.5. The f suffix means float version (vs fabs for double).
Analogy: The distance from zero — always non-negative.

powf(base, exp): Standard C math: raises base to the power exp as floats. powf(2,8)=256. Used to compute qMax = 2^(qMaxBits-1) - 1.
Analogy: 2 to the power of 8 = 256.

2 — TensorConversion.h — Header Explained

Header guard and include

`#ifndef TENSOR_CONVERSION_H`	If this header has NOT been included before, process it. Prevents the compiler from seeing the same declarations twice.
`#define TENSOR_CONVERSION_H`	Mark it as included. Next time #ifndef will be false.
`#include "Tensor.h"`	Bring in tensor_t, shape_t, quantization_t, qtype_t etc. Everything in this file works with tensor_t.

conversionFunction_t — Function Pointer Type

`typedef void (*conversionFunction_t)(tensor_t *inputTensor, tensor_t *outputTensor);`	Creates a new TYPE called conversionFunction_t. void = the function returns nothing. (*conversionFunction_t) = it is a pointer to a function. (tensor_t*, tensor_t*) = the function takes two tensor pointers. Any variable of this type can hold the address of any compatible function. Used to fill the conversionMatrix table.

```
// Example of using conversionFunction_t:
conversionFunction_t fn = convertFloatTensorToSymInt32Tensor;
fn(inputTensor, outputTensor); // calls the function through the pointer
```

Function declarations

Declaration	What it declares
`void convertTensor(tensor_t*, tensor_t*)`	The entry point — the only function you call directly. Looks up and calls the right converter.
`void requantSymInt32Tensor(tensor_t*, tensor_t*)`	SYM_INT32 → SYM_INT32 with a FRESHLY COMPUTED scale (two-pass, dynamic).
`void requantSymInt32TensorToScale(tensor_t*, tensor_t*)`	SYM_INT32 → SYM_INT32 with a PRE-SET scale (one-pass, fixed, saturates by design).
`char *quantTypeToString(qtype_t t)`	Converts a qtype_t enum value to a readable string (e.g. FLOAT32 → "FLOAT32").
`extern conversionFunction_t conversionMatrix[6][6]`	The 6×6 dispatch table. extern means it is defined in TensorConversion.c.

3 — conversionMatrix — The Dispatch Table

The conversionMatrix is a 6×6 table of function pointers. Each cell holds the address of the function that converts from the row's type to the column's type. NULL means same-type (handled separately). unsupportedConversionTypes means the conversion is not implemented.

FROM \ TO	INT32	FLOAT32	SYM_INT32	SYM	ASYM	BOOL
INT32	NULL	→Float	→SymInt32	unsupported	→Asym	unsupported
FLOAT32	→Int32	NULL	→SymInt32	unsupported	→Asym	unsupported
SYM_INT32	extractInt32	→Float32	requant (dynamic)	unsupported	→Asym	unsupported
SYM	unsup	unsup	unsup	NULL	unsup	unsup
ASYM	→Int32	→Float	→SymInt32	unsupported	NULL	unsupported
BOOL	unsup	unsup	unsup	unsup	unsup	NULL

How the matrix is defined in code:

`conversionFunction_t conversionMatrix[6][6] = {`	Declare and initialise a 6×6 array of function pointers.
`[INT32] = {`	Designated initialiser — fill the row for source type INT32. [INT32] uses the enum value (0) as the index.
`[FLOAT32] = convertInt32TensorToFloatTensor,`	Cell [INT32][FLOAT32] = address of the INT32→FLOAT32 converter.
`[SYM_INT32] = convertInt32TensorToSymInt32Tensor,`	Cell [INT32][SYM_INT32] = address of the INT32→SYM_INT32 converter.
`[SYM] = unsupportedConversionTypes,`	Cell [INT32][SYM] = address of the error-print function.
`},`	
`[FLOAT32] = { ... },`	Other rows follow the same pattern.
`[SYM_INT32] = {`	
`[SYM_INT32] = requantSymInt32Tensor,`	The diagonal for SYM_INT32 is requantSymInt32Tensor — not NULL. Same-type is intercepted by convertTensor BEFORE the matrix, so this cell is only reached by direct matrix dispatch.
`},`	
`};`	Closes the matrix initialisation.

★ **Why use a matrix instead of if/else?** With 6 types there are 36 combinations. A nested if/else would need 36 branches. The matrix collapses this to one lookup: fn = matrix[from][to]; fn(in,out). Adding a new type only requires adding one row and one column.

4 — convertTensor — The Entry Point

WHAT IT DOES	The only function you call directly. Reads the types of both tensors, then either copies data (same type) or looks up and calls the right conversion function.
WHY IT EXISTS	Hides all complexity. You never need to know which specific converter to call. Works automatically for any type combination.
HOW TO USE IT	convertTensor(inputTensor, outputTensor) — that is all you write. The input tensor's type and the output tensor's type determine everything else.

`void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	Takes two tensor pointers. The output tensor must already be allocated with the desired quantization type set.
`qtype_t inputDType = inputTensor->quantization->type;`	Read the source type: INT32, FLOAT32, SYM_INT32, etc.
`qtype_t outputDType = outputTensor->quantization->type;`	Read the destination type.
`if (inputDType == outputDType) {`	Same type? Then no format conversion is needed — just copy the bytes.
`convertTensorsWithSameType(inputTensor, outputTensor, inputDType);`	Copies data and also copies scale/zeroPoint metadata if needed.
`} else {`	Different types — need an actual conversion.
`conversionFunction_t conversionFn = conversionMatrix[inputDType][outputDType];`	Table lookup: row=source type, col=dest type. Gets back a function pointer.
`if (conversionFn == NULL) {`	NULL means no converter is registered for this combination. This should never happen if the matrix is correctly filled.
`PRINT_ERROR("No conversion function registered for %s to %s",` `quantTypeToString(inputDType), quantTypeToString(outputDType));`	Print a descriptive error showing both type names.
`exit(1);`	Stop the program. Continuing with NULL would crash anyway.
`}`	
`conversionFn(inputTensor, outputTensor);`	Call the conversion function through the pointer. This is exactly like calling the function directly, but chosen at runtime.
`}`	
`}`	

convertTensorsWithSameType — Internal Same-Type Copy

`static void convertTensorsWithSameType(tensor_t *inputTensor,` `tensor_t *outputTensor, qtype_t qType) {`	static = only visible inside TensorConversion.c. Not callable from outside.
`size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);`	How many values?
`size_t numberOfBytes = calcNumberOfBytesForData(inputTensor->quantization, numberOfElements);`	How many bytes to copy?

`memmove(outputTensor->data, inputTensor->data, numberOfBytes);`	memmove (not memcpy) because input and output may be the SAME buffer. memmove handles overlapping regions safely.
`switch (qType) {`	After copying bytes, also copy the metadata (scale, zeroPoint).
`case SYM_INT32: {`	If it is a symmetric int32 tensor:
`symInt32QConfig_t *inputSymIntQC = inputTensor->quantization->qConfig;` `symInt32QConfig_t *outputSymIntQC = outputTensor->quantization->qConfig;`	Cast to get scale.

`outputSymIntQC->scale = inputSymIntQC->scale;`
`break;`
`}`
	Copy the scale. Without this, the output tensor would have the wrong scale.

`case ASYM: {`	If asymmetric:
`asymQConfig_t *inputAsymQC = inputTensor->quantization->qConfig;` `asymQConfig_t *outputAsymQC = outputTensor->quantization->qConfig;`	

`outputAsymQC->scale = inputAsymQC->scale;`
`outputAsymQC->zeroPoint = inputAsymQC->zeroPoint;`
`break;`
`}`
	Copy scale and zeroPoint — both are needed for ASYM.

`default: break;` `}`	For INT32, FLOAT32, BOOL: no extra metadata to copy.
`}`	

5 — Helper Functions

zeroTensorData

WHAT IT DOES	Fills every byte of a tensor's data buffer with 0.
WHY IT EXISTS	Before writing converted values, the output buffer must be clean. memset is faster than looping manually.
HOW TO USE IT	Called at the start of conversion functions that build up output incrementally.

`void zeroTensorData(tensor_t *tensor) {`	
`size_t numberOfElements = calcNumberOfElementsByTensor(tensor);`	How many values does this tensor have?
`size_t bytesPerElement = calcBytesPerElement(tensor->quantization);`	How many bytes per value?
`memset(tensor->data, 0, numberOfElements * bytesPerElement);`	memset fills every byte with 0. numberOfElements * bytesPerElement = total bytes to zero.
`}`	

copyDimsAndSparsityToTensor

WHAT IT DOES	Copies the shape pointer and sparsity data from input to output tensor.
WHY IT EXISTS	Shape does not change during conversion (FLOAT32→SYM_INT32 has the same dimensions). This function ensures the output tensor has the correct shape metadata.
HOW TO USE IT	Called at the end of most conversion functions.

`void copyDimsAndSparsityToTensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`outputTensor->shape = inputTensor->shape;`	Share the shape pointer — both tensors now point to the same shape_t. Safe because shape is not modified by conversion.
`if (inputTensor->sparsity) {`	Only copy sparsity if the input has one (non-NULL).
`memcpy(outputTensor->sparsity, inputTensor->sparsity, sizeof(sparsity_t));`	Deep copy the sparsity struct.
`}`	
`}`	

6 — INT32 Conversions

convertInt32TensorToFloatTensor

WHAT IT DOES	Converts every INT32 integer into a FLOAT32. Cast each int32_t to float.
WHY IT EXISTS	INT32 tensors are raw integers with no scale — converting to float is just a type cast.
HOW TO USE IT	Called when matrix[INT32][FLOAT32] is dispatched.

`void convertInt32TensorToFloatTensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);`	How many values?
`int32_t inputData[numberOfElements];`	VLA: stack-allocated array to hold the integers decoded from raw bytes.
`float outputData[numberOfElements];`	VLA: stack-allocated array to hold the converted floats.
`readBytesAsInt32Array(numberOfElements, inputTensor->data, inputData);`	Decode the raw bytes of the input buffer into proper int32_t values.
`zeroTensorData(outputTensor);`	Clear the output buffer before writing.
`for (size_t i = 0; i < numberOfElements; i++) {`	Loop over every element.
`outputData[i] = (float)inputData[i];`	(float) cast converts the integer to its floating-point equivalent. No scale involved — INT32 is raw, so 127 becomes 127.0f.
`}`	
`writeFloatArrayToByteArray(numberOfElements, outputData, outputTensor->data);`	Pack the float values into the output tensor's raw byte buffer.
`copyDimsAndSparsityToTensor(inputTensor, outputTensor);`	Copy shape and sparsity to the output.
`}`	

convertInt32TensorToSymInt32Tensor

WHAT IT DOES	Copies the raw int32 bytes directly and sets the output scale to 1.
WHY IT EXISTS	INT32 has no scale — treating it as SYM_INT32 with scale=1 means the integer values are preserved and scale=1 means real_value = mantissa × 1 = mantissa.
HOW TO USE IT	Used to wrap raw integers in the SYM_INT32 quantization format.

`void convertInt32TensorToSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);`	
`symInt32QConfig_t *outputSymInt32QConfig = outputTensor->quantization->qConfig;`	Get the output's quantization config.
`outputSymInt32QConfig->scale = 1;`	Set scale=1: real_value = mantissa × 1 = mantissa. The values are unchanged.
`memcpy(outputTensor->data, inputTensor->data, numberOfElements * sizeof(int32_t));`	Copy raw bytes directly — no conversion needed, just a byte-for-byte copy.
`}`	

convertInt32TensorToAsymTensor

WHAT IT DOES	Converts INT32 to ASYM by computing scale and zeroPoint from the data range, then quantizing each value.
WHY IT EXISTS	ASYM needs to cover the full [min, max] range of the input. Scale and zeroPoint are computed from the actual data.
HOW TO USE IT	Used when matrix[INT32][ASYM] is dispatched.

`void convertInt32TensorToAsymTensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);`	
`int32_t min = findMinInt32(inputTensor->data, numberOfElements);`	Scan all values to find the minimum.
`int32_t max = findMaxInt32(inputTensor->data, numberOfElements);`	Scan all values to find the maximum.
`asymQConfig_t *linearQConfig = outputTensor->quantization->qConfig;`	Get the output's ASYM config.
`int32_t qMax = pow(2, linearQConfig->qBits) - 1;`	Compute qMax: for 8-bit → 2^8-1=255. For 4-bit → 2^4-1=15.
`float scale = (float)(max - min) / (float)qMax;`	Scale = data range / representable range. Maps [min,max] onto [0,qMax].
`int16_t zeroPoint = (int16_t)roundByMode((float)min / scale, ...);`	zeroPoint is the integer that represents the float value 'min'. real_value = (integer + zeroPoint) * scale → min = (0 + zeroPoint) * scale → zeroPoint = min/scale.
`for (size_t elementIndex = 0; elementIndex < numberOfElements; elementIndex++) {`	
`int32_t inputElement = readBytesAsInt32(&inputTensor->data[elementIndex * sizeof(int32_t)]);`	Read one int32 value from raw bytes.
`outputElements[elementIndex] =`	Compute the output integer:
`roundByMode(clamp((float)inputElement / scale - (float)zeroPoint, 0.f, qMax-1), ...);`	inputElement/scale converts to float range. Subtracting zeroPoint shifts to start at 0. clamp ensures the result is within [0, qMax-1]. roundByMode converts to integer.
`}`	
`linearQConfig->scale = scale; linearQConfig->zeroPoint = zeroPoint;`	Store the computed scale and zeroPoint into the output config.
`byteConversion(outputElement, 32, outputTensor->data, linearQConfig->qBits, n);`	Pack the int32 values into qBits-wide packed format in the output buffer.
`copyDimsAndSparsityToTensor(inputTensor, outputTensor);`	
`}`	

7 — FLOAT32 Conversions

convertFloatTensorToInt32Tensor

WHAT IT DOES	Converts every FLOAT32 value to INT32 by truncating (casting to int32_t).
WHY IT EXISTS	Inverse of convertInt32TensorToFloatTensor. Truncates decimal part.
HOW TO USE IT	Called when matrix[FLOAT32][INT32] is dispatched.

`void convertFloatTensorToInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);`	
`float inputData[numberOfElements]; int32_t outputData[numberOfElements];`	Stack arrays for reading input floats and holding output integers.
`readBytesAsFloatArray(numberOfElements, inputTensor->data, inputData);`	Decode raw bytes into float array.
`zeroTensorData(outputTensor);`	Clear output buffer.
`for (size_t i = 0; i < numberOfElements; i++) {`	
`outputData[i] = (int32_t)inputData[i];`	(int32_t) cast truncates the decimal: 3.7 → 3, -2.9 → -2.
`}`	
`writeInt32ArrayToByteArray(numberOfElements, outputData, outputTensor->data);`	Pack integers back into raw bytes.
`copyDimsAndSparsityToTensor(inputTensor, outputTensor);`	
`}`	

convertFloatTensorToSymInt32Tensor — THE key training conversion

WHAT IT DOES	Quantizes every float to SYM_INT32 by computing a scale from absMax, then converting each float: out[i] = round(clamp(float[i]/scale, qMin, qMax)).
WHY IT EXISTS	This is how float weights become quantized integers for FQT training. The scale is derived from the actual data so no clamping loss occurs.
HOW TO USE IT	Called when matrix[FLOAT32][SYM_INT32] is dispatched — the most important conversion in the whole system.

`void convertFloatTensorToSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);`	
`float absMax = findAbsMaxFloat(inputTensor->data, numberOfElements);`	Scan all floats to find the largest absolute value. This determines the scale.
`symInt32QConfig_t *symInt32QC = outputTensor->quantization->qConfig;`	Get the output's config — provides qMaxBits and roundingMode.
`uint8_t qMaxBits = symInt32QC->qMaxBits;`	How many bits? E.g. 8 → qMax=127.
`const float qMax = powf(2, (float)qMaxBits - 1) - 1;`	qMax = 2^(qMaxBits-1) - 1. For 8-bit: 2^7-1=127.
`const float qMin = -powf(2, (float)qMaxBits - 1);`	qMin = -2^(qMaxBits-1). For 8-bit: -128.

`float scale;`	
`if (absMax == 0.f) {`	All-zeros tensor: cannot divide by absMax. Use scale=1 as safe default.
`scale = 1.f;` `} else {`	
`scale = absMax / qMax;`	The largest float maps exactly to qMax. Every other float maps to a smaller integer. No clamping.
`}`	
`outputSymInt32QC->scale = scale;`	Store the computed scale into the output config.
`int32_t *outputInt32 = (int32_t *)outputTensor->data;`	Treat the output byte buffer as an int32_t array. Safe because each SYM_INT32 element is 4 bytes = sizeof(int32_t).
`float *inputFloat = (float *)inputTensor->data;`	Treat the input byte buffer as a float array.
`for (size_t i = 0; i < numberOfElements; i++) {`	
`outputInt32[i] =` `roundByMode(clamp(inputFloat[i] / scale, qMin, qMax), outputSymInt32QC->roundingMode);`	Compute and store the quantized integer: Step 1: inputFloat[i]/scale → scales the float to integer range. Step 2: clamp → ensures result stays within [qMin, qMax]. Step 3: roundByMode → converts float to int using the configured rounding strategy.
`}`	
`}`	

```
// Example: float=0.85, absMax=1.0, qMax=127, scale=1.0/127=0.00787
// 0.85 / 0.00787 = 107.9 → clamp(107.9,-128,127) = 107.9 → round = 108
// Stored: 108. Recovered: 108 × 0.00787 = 0.850 (≈ original)
```

convertFloatTensorToAsymTensor

WHAT IT DOES	Quantizes floats to ASYM by computing scale and zeroPoint from [min, max] range.
WHY IT EXISTS	Asymmetric is better for non-symmetric distributions (e.g. post-ReLU activations that are all positive).
HOW TO USE IT	Called when matrix[FLOAT32][ASYM] is dispatched.

`void convertFloatTensorToAsymTensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`float min = findMinFloat(inputTensor->data, numberOfElements);`	Find minimum float value.
`float max = findMaxFloat(inputTensor->data, numberOfElements);`	Find maximum float value.
`float qMax = pow(2, asymQConfig->qBits);`	ASYM uses unsigned range [0, qMax]. For 8-bit: qMax=256.
`if (min == max) {`	Edge case: all values are equal — cannot compute a meaningful scale.
`scale = min; zeroPoint = 1;` `} else {`	Fallback: scale=min, zeroPoint=1. Prevents division by zero.
`scale = (max - min) / qMax;`	Maps the float range [min,max] onto [0,qMax].
`zeroPoint = roundByMode(min / scale, ...);`	Integer that represents the float value 'min'.

`}`	
`for (size_t i = 0; i < numberOfElements; i++) {`	
`outputElements[i] = roundByMode(clamp(inputFloat[i]/scale - zeroPoint, 0, qMax-1), ...);`	Convert each float: divide by scale, subtract zeroPoint, clamp, round.
`}`	
`byteConversion(outputElement, 32, outputTensor->data, asymQConfig->qBits, n);`	Repack from 32-bit integers to qBits-wide packed format.
`}`	

8 — SYM_INT32 Conversions

extractInt32TensorFromSymInt32Tensor

WHAT IT DOES	Copies raw int32 bytes from SYM_INT32 to INT32. IGNORES the scale.
WHY IT EXISTS	Sometimes you want just the raw integer mantissas without dequantizing. The comment says 'Scale is ignored!' deliberately.
HOW TO USE IT	Called when matrix[SYM_INT32][INT32] is dispatched.

```
// Important: Scale is ignored!
void extractInt32TensorFromSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
    size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);
    size_t bytesPerElement = sizeof(int32_t);                    // 4 bytes per element.
    int32_t inputAsInt32[numberOfElements];                      // Stack array for decoded integers.
    readBytesAsInt32Array(numberOfElements, inputTensor->data, inputAsInt32);   // Decode raw bytes to int32 array.
    memcpy(outputTensor->data, inputAsInt32, numberOfElements * bytesPerElement);   // Copy integers to output — no scale applied.
}
```

convertSymInt32TensorToFloat32Tensor

WHAT IT DOES	Dequantizes SYM_INT32 back to FLOAT32. Each element: float = int32 × scale.
WHY IT EXISTS	After quantized training, you may need float tensors for comparison or export.
HOW TO USE IT	Called when matrix[SYM_INT32][FLOAT32] is dispatched.

```
void convertSymInt32TensorToFloat32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
    int32_t *inputAsInt32 = (int32_t *)inputTensor->data;   // Treat input raw bytes as int32 array.
    float output[numberOfValues];                           // Stack array for output floats.
    symInt32QConfig_t *inputSymInt32QConfig = inputTensor->quantization->qConfig;   // Get the input's config to read the scale.
    float scale = inputSymInt32QConfig->scale;              // The scale that was stored when this tensor was quantized.
    for (size_t i = 0; i < numberOfValues; i++) {
        output[i] = (float)inputAsInt32[i] * scale;         // Dequantize: real_value = mantissa × scale. This recovers an approximation of the original float.
    }
    memcpy(outputTensor->data, output, numberOfValues * sizeof(float));   // Write float values to output buffer.
}
```

9 — requantSymInt32Tensor — Dynamic Two-Pass Requantization

This is the most important function for FQT training. It converts SYM_INT32 → SYM_INT32 but computes a completely fresh scale from the actual data range. No clamping ever occurs.

WHAT IT DOES	Two-pass: Pass A scans all values to find absMax and compute a new scale. Pass B uses the new scale to re-quantize every element.
WHY IT EXISTS	After a gradient update, the weight magnitudes change. The old scale no longer fits. A fresh scale ensures the new values are represented accurately without loss.
HOW TO USE IT	Wired into conversionMatrix[SYM_INT32][SYM_INT32]. Reached only via direct matrix dispatch (convertTensor short-circuits same-type before the matrix).

★ **Why two passes?** You must know the new scale before you can write any output value. But to compute the scale you must read ALL values first. You cannot do both in one pass.

`void requantSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`size_t numberOfElements = calcNumberOfElementsByTensor(inputTensor);`	
`symInt32QConfig_t *inputSymInt32QC = inputTensor->quantization->qConfig;`	Get input's config — provides inScale.
`symInt32QConfig_t *outputSymInt32QC = outputTensor->quantization->qConfig;`	Get output's config — provides qMaxBits, roundingMode; scale will be written here.
`float inScale = inputSymInt32QC->scale;`	LATCH the input scale NOW, before anything is written. Critical for in-place use (input==output): once we write outputSymInt32QC->scale, inputSymInt32QC->scale would also change (they alias the same config). Saving it here in a local variable keeps it safe.
`const float qMax = powf(2, (float)outputSymInt32QC->qMaxBits - 1) - 1;`	Compute qMax from the output's bit width. E.g. 8-bit → 127.
`const float qMin = -powf(2, (float)outputSymInt32QC->qMaxBits - 1);`	Compute qMin. E.g. 8-bit → -128.
`int32_t *inputInt32 = (int32_t *)inputTensor->data;`	Treat input raw bytes as int32 array.
`int32_t *outputInt32 = (int32_t *)outputTensor->data;`	Treat output raw bytes as int32 array.
`/* pass A: absmax over dequantized values — reads only (alias-safe) */`	
`float absMax = 0.f;`	Start with absMax=0. Will be updated for each element.
`for (size_t i = 0; i < numberOfElements; i++) {`	Loop over every element.
`float dequant = fabsf((float)inputInt32[i] * inScale);`	Dequantize: real_value = mantissa × inScale. fabsf() takes the absolute value (always positive).
`if (dequant > absMax) {`	Is this the new largest value?
`absMax = dequant;`	Update absMax.
`}`	
`}`	End of Pass A.
`float scale;`	
`if (absMax == 0.f) {`	All values are zero:
`scale = 1.f;`	Default scale=1 to avoid division by zero.

`} else {`	
`scale = absMax / qMax;`	New scale: the largest real value maps exactly to qMax. Example: absMax=1.27, qMax=127 → scale=0.01.
`}`	
`outputSymInt32QC->scale = scale;`	Write the new scale into the output config.
`/* pass B: same-index read-then-write — in-place safe */`	
`for (size_t i = 0; i < numberOfElements; i++) {`	
`outputInt32[i] = roundByMode(`	Re-quantize element i:
`clamp(((float)inputInt32[i] * inScale) / scale, qMin, qMax),`	Step 1: (mantissa × inScale) = real_value. Step 2: real_value / scale = new_mantissa (float). Step 3: clamp to [qMin, qMax] (never triggers — scale was derived from absMax).
`outputSymInt32QC->roundingMode);`	Step 4: round to integer using HALF_AWAY or SR_HALF_AWAY.
`}`	End of Pass B.
`}`	

```
// Full numeric example:
// Inputs: mantissas=[100,-80,50,127], inScale=0.01
// Real values: [1.0, -0.8, 0.5, 1.27]
//
// Pass A: absMax = max(1.0, 0.8, 0.5, 1.27) = 1.27
// newScale = 1.27 / 127 = 0.01
//
// Pass B: out[0] = round(clamp(1.0 / 0.01, -128, 127)) = round(100.0) = 100
// out[1] = round(clamp(-0.8 / 0.01, -128, 127)) = round(-80.0) = -80
// out[2] = round(clamp(0.5 / 0.01, -128, 127)) = round(50.0) = 50
// out[3] = round(clamp(1.27 / 0.01, -128, 127)) = round(127.0) = 127
// newScale stored in output qConfig = 0.01
```

10 — requantSymInt32TensorToScale — Fixed-Scale One-Pass

Same conversion as requantSymInt32Tensor but the target scale is set by the CALLER before the call. The function never modifies the scale. Values outside the representable range are CLAMPED — this is intentional.

WHAT IT DOES	One-pass: for each element, convert using the pre-set targetScale. Clamp values that exceed the range. Never modifies the output scale.
WHY IT EXISTS	When you know the target scale in advance (e.g. a shared scale across a layer), you do not want the scale recomputed from this tensor. Clamping is the correct behaviour for values out of range.
HOW TO USE IT	Set outputTensor->quantization->qConfig->scale before calling. NOT in conversionMatrix — call directly when you need fixed-scale requantization.

`void requantSymInt32TensorToScale(tensor_t *inputTensor, tensor_t *outputTensor) {`	

`symInt32QConfig_t *inputSymInt32QC = inputTensor->quantization->qConfig;`
`symInt32QConfig_t *outputSymInt32QC = outputTensor->quantization->qConfig;`
`float inScale = inputSymInt32QC->scale;`
`float targetScale = outputSymInt32QC->scale;`
	Latch input scale (same in-place safety reason as requantSymInt32Tensor).
	Read the pre-set target scale. Must be set by caller before this call.

`if (!(targetScale > 0.f)) {`
`PRINT_ERROR("requantSymInt32TensorToScale: target scale must be pre-set and > 0...");`
`exit(1);`
`}`
	NaN-robust guard. !(x > 0) is true when x <= 0 OR when x is NaN. (x <= 0) would miss NaN because NaN comparisons always return false.
	Stop immediately.

`const float qMax = powf(2, (float)outputSymInt32QC->qMaxBits - 1) - 1;`
`const float qMin = -powf(2, (float)outputSymInt32QC->qMaxBits - 1);`
`int32_t *inputInt32 = (int32_t *)inputTensor->data;`
`int32_t *outputInt32 = (int32_t *)outputTensor->data;`	

`for (size_t i = 0; i < numberOfElements; i++) {`
`outputInt32[i] =`
`roundByMode(`
`clamp(((float)inputInt32[i] * inScale) / targetScale, qMin, qMax),`
`outputSymInt32QC->roundingMode);`
`}`
	Single pass — no absMax needed.
	Compute output integer:
	Same formula as requantSymInt32Tensor but using targetScale (fixed). If a value exceeds the range, clamp brings it to qMin or qMax — this is INTENTIONAL SATURATION, not an error.

`}`	

Feature	requantSymInt32Tensor	requantSymInt32TensorToScale

`Scale`	COMPUTED from data (dynamic)	PRE-SET by caller (fixed)
`Passes`	2 (read-all then write-all)	1 (read-write each element)
`Clamping`	Never — scale derived from absMax	YES by design — saturation is correct
`In conversionMatrix`	YES — [SYM_INT32][SYM_INT32]	NO — call directly
`When to use`	After gradient update (weights changed)	Layer-epilogue with known shared scale

11 — ASYM Conversions

convertAsymTensorToInt32Tensor

WHAT IT DOES	Unpacks ASYM N-bit integers to int32 and adds the zeroPoint to each.
WHY IT EXISTS	ASYM stores values as unsigned integers offset by zeroPoint. Adding zeroPoint recovers the signed integer representation.
HOW TO USE IT	Called when matrix[ASYM][INT32] is dispatched.

`void convertAsymTensorToInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`asymQConfig_t *asymQConfig = inputTensor->quantization->qConfig;`	Read ASYM config.
`int16_t zeroPoint = asymQConfig->zeroPoint;`	The offset to add.
`uint8_t dataOut[numberOfElements * sizeof(int32_t)];`	Buffer for unpacked int32 bytes.
`memset(dataOut, 0, numberOfElements * sizeof(int32_t));`	Zero the buffer before byteConversion.
`byteConversion(inputTensor->data, asymQConfig->qBits, dataOut, 32, numberOfElements);`	Expand N-bit packed values → 32-bit integers.
`readBytesAsInt32Array(numberOfElements, dataOut, outputElements);`	Decode the 32-bit byte buffer into an int32 array.
`for (size_t elementIndex = 0; elementIndex < numberOfElements; elementIndex++) {`	
`outputElements[elementIndex] = outputElements[elementIndex] + zeroPoint;`	Add zeroPoint: converts unsigned ASYM integer back to signed integer. real_int = asym_int + zeroPoint.
`}`	
`writeInt32ArrayToByteArray(numberOfElements, outputElements, outputTensor->data);`	
`copyDimsAndSparsityToTensor(inputTensor, outputTensor);`	
`}`	

convertAsymTensorToFloatTensor

WHAT IT DOES	Unpacks ASYM integers and fully dequantizes to float: float = (int + zeroPoint) × scale.
WHY IT EXISTS	Full dequantization to recover approximate original float values.
HOW TO USE IT	Called when matrix[ASYM][FLOAT32] is dispatched.

`void convertAsymTensorToFloatTensor(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`zeroTensorData(outputTensor);`	Clear output.
`int16_t zeroPoint = asymQConfig->zeroPoint;`	Offset.
`byteConversion(inputTensor->data, asymQConfig->qBits, (uint8_t*)inputInt, 32, n);`	Unpack N-bit → int32.
`float *outputElements = (float *)outputTensor->data;`	Treat output buffer as float array.

```
for (size_t elementIndex = 0; elementIndex < numberOfElements; elementIndex++) {
outputElements[elementIndex] =
((float)inputInt[elementIndex] + (float)zeroPoint) * asymQConfig->scale;

}
copyDimsAndSparsityToTensor(inputTensor, outputTensor);
}
```

Dequantize:

real_value = (asym_int + zeroPoint) × scale. Adding zeroPoint shifts back to signed range; multiplying by scale converts to float.

convertAsymTensorToSymInt32Tensor

WHAT IT DOES	Converts ASYM → SYM_INT32 by unpacking, adding zeroPoint, and copying the scale.
WHY IT EXISTS	Bridges asymmetric activations into the symmetric training world.
HOW TO USE IT	Called when matrix[ASYM][SYM_INT32] is dispatched.

```
void convertAsymTensorToSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
size_t bitsPerInputElement = calcBitsPerElement(inputTensor->quantization);
int16_t zeroPoint = inputAsymQConfig->zeroPoint;
```

How many bits per ASYM value (e.g. 8)?

```
byteConversion(inputTensor->data, bitsPerInputElement, (uint8_t*)inputAsInt32, 32, n);
for (size_t i = 0; i < numberOfElements; i++) {
inputAsInt32[i] += zeroPoint;

}
```

Unpack N-bit ASYM values to int32 array.

Add zeroPoint to convert from unsigned ASYM range to signed SYM range.

```
memcpy(outputTensor->data, inputAsInt32, numberOfElements * sizeof(int32_t));
outputSymInt32QConfig->scale = inputAsymQConfig->scale;
}
```

Copy the int32 values to the output buffer.

The scale is the same — only the integer representation changed.

12 — quantTypeToString / unsupportedConversionTypes

quantTypeToString

WHAT IT DOES	Converts a qtype_t enum value to a human-readable C string.
WHY IT EXISTS	Error messages need readable type names. Without this you would print numbers (0,1,2...) instead of names.
HOW TO USE IT	Called in PRINT_ERROR and convertTensor's NULL-check error message.

`char *quantTypeToString(qtype_t t) {`	Returns a pointer to a string literal.
`switch (t) {`	Branch on the enum value.

`case INT32: return "INT32";`
`case FLOAT32: return "FLOAT32";`
`case SYM_INT32:return "SYMINT32";`
`case SYM: return "SYM";`
`case ASYM: return "ASYM";`
`case BOOL: return "BOOL";`
	enum INT32=0 → return the string "INT32".

`default: return "UNKNOWN";`	Catch-all — should never happen if the enum is used correctly.
`}`	
`}`	

unsupportedConversionTypes

WHAT IT DOES	Prints an error showing which conversion was attempted, then stops the program.
WHY IT EXISTS	The conversionMatrix must have a function in every cell — even unsupported ones. This function fills those cells so attempting an unsupported conversion fails clearly rather than crashing with a NULL dereference.
HOW TO USE IT	Never called directly. Placed in conversionMatrix cells where no real converter exists.

`void unsupportedConversionTypes(tensor_t *inputTensor, tensor_t *outputTensor) {`	
`qtype_t inputQType = inputTensor->quantization->type;`	Read source type for the error message.
`qtype_t outputQType = outputTensor->quantization->type;`	Read destination type for the error message.
`PRINT_ERROR("Conversion from %s to %s is not supported",` `quantTypeToString(inputQType), quantTypeToString(outputQType));`	Print a clear message showing exactly what conversion was attempted.
`exit(1);`	Stop the program.
`}`	

13 — _Static_assert Guard

```
_Static_assert(BOOL + 1 == 6,

"extend conversionMatrix when adding qtype_t
entries");
```

_Static_assert is a compile-time check. If the condition is false, the compiler refuses to build the project.

The message that appears if the check fails. BOOL is the last value in the qtype_t enum. BOOL+1 should equal 6 (the number of types: INT32=0, FLOAT32=1, SYM_INT32=2, SYM=3, ASYM=4, BOOL=5). If someone adds a 7th type to the enum, BOOL becomes 6, BOOL+1=7 ≠ 6, and the compiler refuses to build — forcing the developer to also extend the matrix.

■ **Why this matters:** If you add a new quantization type (e.g. FP8) to the qtype_t enum without updating conversionMatrix, the matrix would still be [6][6] but the new type would index out of bounds — silent memory corruption. _Static_assert catches this at compile time, not at runtime.

14 — Quick Reference Card

Function	From	To	Key behaviour
convertTensor	any	any	Entry point — matrix lookup or same-type copy
convertTensorsWithSameType	any	same	memmove + copy scale metadata
zeroTensorData	—	—	memset tensor data to 0
copyDimsAndSparsityToTensor	—	—	Share shape pointer, copy sparsity
convertInt32TensorToFloatTensor	INT32	FLOAT32	(float) cast, no scale
convertInt32TensorToSymInt32Tensor	INT32	SYM_INT32	memcpy + set scale=1
convertInt32TensorToAsymTensor	INT32	ASYM	Compute scale+zeroPoint, byteConversion
convertFloatTensorToInt32Tensor	FLOAT32	INT32	(int32_t) cast, truncates
convertFloatTensorToSymInt32Tensor	FLOAT32	SYM_INT32	absMax→scale, round(clamp(x/scale))
convertFloatTensorToAsymTensor	FLOAT32	ASYM	[min,max]→scale+zeroPoint, byteConversion
extractInt32TensorFromSymInt32Tensor	SYM_INT32	INT32	Copy bytes, IGNORE scale
convertSymInt32TensorToFloat32Tensor	SYM_INT32	FLOAT32	Dequant: int × scale
requantSymInt32Tensor	SYM_INT32	SYM_INT32	2-pass, dynamic scale, no clamp
requantSymInt32TensorToScale	SYM_INT32	SYM_INT32	1-pass, fixed scale, saturates
convertSymInt32TensorToAsymTensor	SYM_INT32	ASYM	Dequant→float→ASYM quantize
convertAsymTensorToInt32Tensor	ASYM	INT32	Unpack + add zeroPoint
convertAsymTensorToFloatTensor	ASYM	FLOAT32	Unpack + (int+zp)*scale
convertAsymTensorToSymInt32Tensor	ASYM	SYM_INT32	Unpack + add zeroPoint + copy scale
quantTypeToString	—	—	enum → readable string for error messages
unsupportedConversionTypes	any	any	Print error + exit(1)